Subject

Object Oriented programming

and

Computer Graphics

Unit-I

Introduction to OOP Concepts

and

Control Structure

Savitribai Phule Pune University Second Year of Computer Engineering and Computer Science and Engineering (2024 Course)		
PCC-202-COM : Object Oriented programming and Computer Graphics		
Teaching /scheme	**Credits**	**Examination Scheme**
Theory : 03 Hours/Week	03	**CCE** : 30 Marks **End-Semester :** 70 Marks

Contents

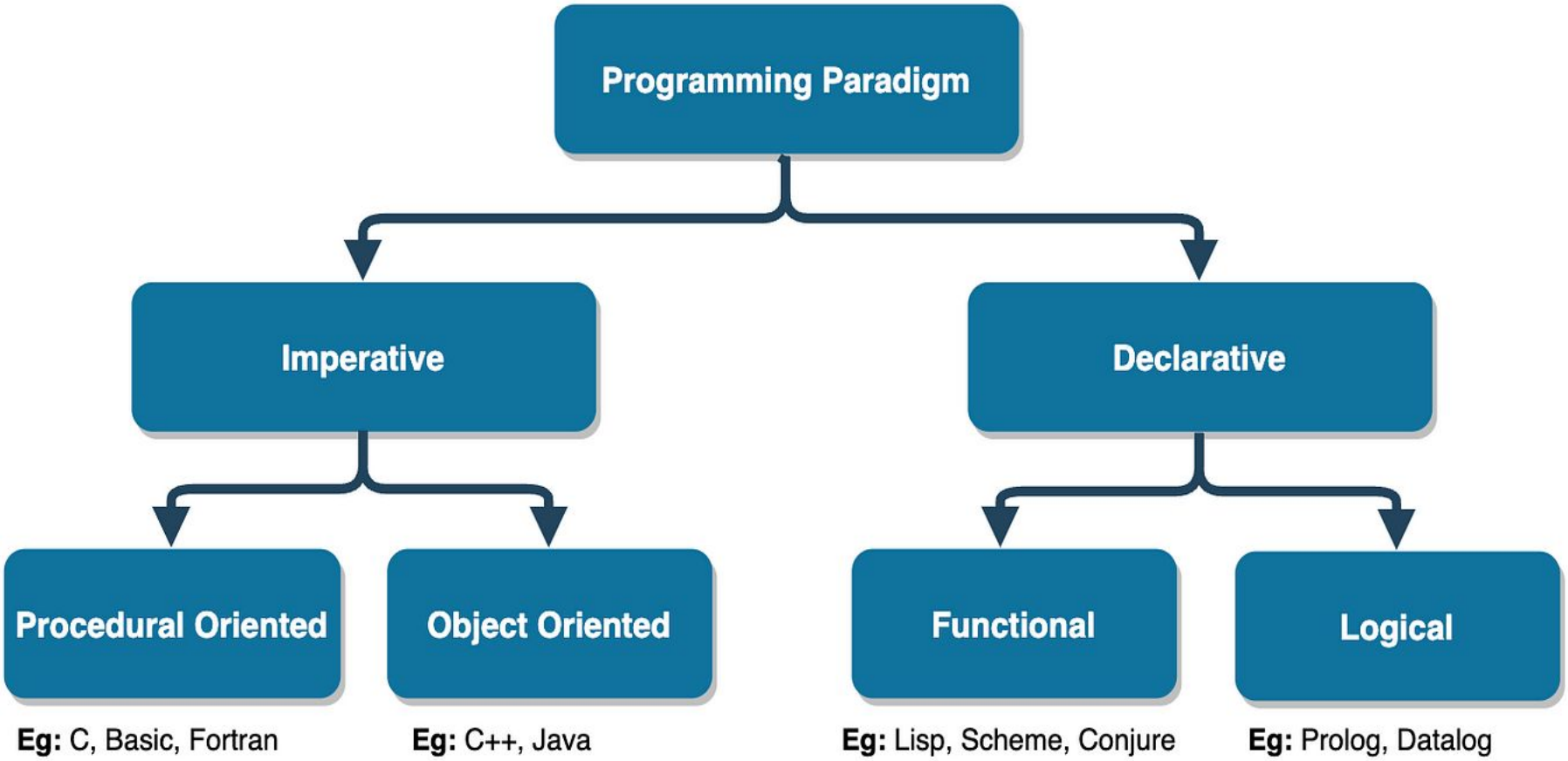
- **Programming paradigms-**
  - Introduction to programming paradigms,
  - Introduction to four main Programming paradigms- procedural, object oriented, functional, and logic & rule based.
  - Need of object oriented programming,
- **Fundamentals of object-oriented programming:**
  - Namespaces,
  - objects,
  - classes,
  - data members,
  - methods,
  - messages,
  - data encapsulation,
  - data abstraction and information hiding,
  - inheritance,
  - polymorphism.
  - Benefits of OOP,
  - Java as object oriented programming language.

- **Overview of Java Language:**
  - Simple Java program structure: documentation section, package statement, import statements, class definition, main method class.
  - Implementing Java Program,
  - JVM,
  - Data types, Primitive Types vs. Reference type,
  - floating point numbers,
  - Operators and expressions,
  - Java Class Libraries,
  - Typical Java Development Environment, and Memory Concepts.
- **Control Statements:**
  - Selection Statements: if, if-else, nested if-else,
  - Iteration Statements: do, while, for, for-each statement,
  - Break, and continue statements
- **Case Study:** Analyze the object-oriented features in Java with other object oriented programming languages.

What is a Programming Paradigm?

- Programming paradigms are fundamental styles or approaches to programming, providing a way to structure and organize code.
- They are not specific languages but rather represent different ways of thinking about and solving problems using a programming language.
- Each paradigm has its own set of principles and techniques for writing, organizing, and executing programs.
- Apart from the variety of programming languages available, there are several paradigms that address different demands and challenges in software development. These paradigms are discussed below:

Introduction to four main Programming paradigms



1) Imperative Paradigm

- In the Imperative type of Language, The programmers instruct the machine **How** to perform a task by writing step-by-step instructions including variables, conditions, loops etc.
- **Example:** When you are hungry, you go to the kitchen and make your own food by adding the necessary ingredients and following the cooking instructions.
- **Programming Example:** Languages like C, C++, and Java belong to Imperative Paradigm.
- Types:

  1) Procedural programming paradigm

  2) Object oriented programming

1. Procedural Programming:

- Procedural programming is a paradigm where the program is structured around procedures or functions that manipulate data.
- It focuses on step-by-step instructions and emphasizes code reusability through the use of functions.
- Eg. C and Pascal

2. Object-Oriented Programming (OOP):

- Object-Oriented Programming revolves around the concept of objects, which are instances of classes.
- It organizes code into objects that encapsulate data and behavior.
- OOP promotes modularity, reusability, and allows for concepts such as inheritance, polymorphism, and encapsulation.
- Eg. Java, C++, and Python

2. Declarative programming paradigm

- In the Declarative type of Language, The programmers instruct the machine **What** to perform without any instructions. The programmer asks for the desired result.
- **Example:** When you are hungry, you order food from a restaurant. You don't know how the food is made. You just asked for a desired food and it's delivered to you.
- **Programming Example:** SQL (Structured Query Language) belongs to the declarative language.
- Types:

  1) Functional programming paradigm

  2) Logical and rule based programming

1. Functional Programming (FP):

- Functional Programming treats computation as the evaluation of mathematical functions and avoids changing state and mutable data.
- It emphasizes immutability, pure functions, and higher-order functions.
- FP languages focus on expressing computations as the evaluation of expressions.
- E.g. Haskell, Lisp, and Erlang

2. Logical or rule-based programming

- Logical or rule-based programming is a paradigm where programs are constructed using facts and rules to represent knowledge and relationships within a system.
- It can be termed as abstract model of computation.
- It would solve logical problems like puzzles, series etc. In logic programming we have a knowledge base which we know before and along with the question and knowledge base which is given to machine, it produces result.
- e.g., Prolog

Need of object oriented programming

- **1. Managing Complexity:**
- Large and complex software can be difficult to understand and maintain. OOP helps by breaking down the system into smaller, manageable objects, each with its own data and methods (functions that operate on the data).

- **2. Code Reusability:**
- OOP promotes code reuse through concepts like inheritance, where new classes can inherit properties and behaviors from existing ones.
- This reduces redundancy and development time, as you don't have to rewrite code for similar functionalities.

- **3. Data Security:**
- OOP allows for encapsulation, where data is bundled within an object and protected from outside access.
- Only the methods of the object can interact with its data, preventing accidental or unauthorized modification.

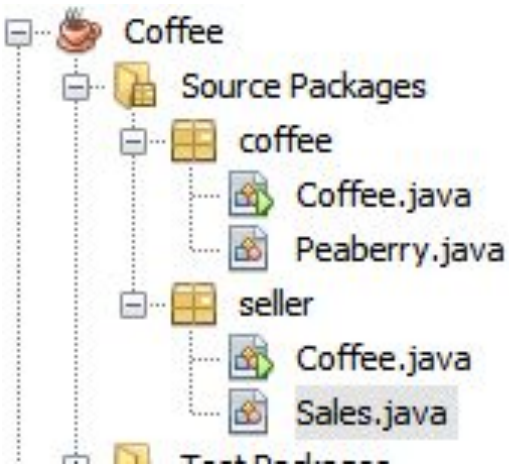
- **4. Real-World Modeling:**
- OOP allows for the modeling of real-world entities as objects, making it easier to represent and simulate complex systems.
- This natural mapping between the real world and the code can simplify the development process and improve understanding.

- **5. Flexibility and Scalability:**
- OOP provides flexibility through inheritance and polymorphism, allowing for the creation of versatile and adaptable systems.
- As the software evolves, OOP makes it easier to add new features or modify existing ones without affecting other parts of the system.

Fundamentals of object-oriented programming

Namespace

- A namespace is a fundamental concept in programming and other fields that provides a hierarchical or structured way to organize and group related identifiers (like variables, functions, classes, etc.) under a single name.
- It prevents naming conflicts and provides a scope for these identifiers, making it easier to manage and reuse code, especially in larger projects or when integrating code from multiple sources.
-



- In Java Object-Oriented Programming (OOP), the concept of "namespace" is primarily implemented through **packages**.
- While some other languages use a dedicated namespace keyword, Java achieves the same organizational and naming conflict-prevention goals using packages.
- Here's how packages function as namespaces in Java OOP:
- **Organization and Grouping:**
- Packages provide a hierarchical structure for organizing related classes, interfaces, enumerations, and annotations into logical units.
- **Preventing Naming Conflicts:**
- The most crucial role of packages as namespaces is to prevent naming collisions.
- Two classes with the same name can exist in different packages without conflict because their fully qualified names (including the package name) will be unique.
- For example, **java.util.List** and **java.awt.List** are distinct classes despite sharing the name "List."

OBJECTS

- In Java, an object is a fundamental concept in Object-Oriented Programming (OOP) and represents a concrete instance of a class.
- It can be understood as a real-world entity that possesses a state (data) and exhibits behavior (methods).
- An object is any entity that has a **state** and **behavior**. For example, a **bicycle** is an object. It has
- **States**: idle, first gear, etc
- **Behaviors**: braking, accelerating, etc.

Java Class

- A class is a **blueprint for the object**.
- Before we create an object, we first need to define the class.
- We can think of the class as a sketch (prototype).

class	objects
Fruit	Apple Banana Mango
Car	Volvo Audi Toyota

Java Classes/Objects

How to Create a Class?

- To create a class, use the keyword class:

Main.java

Create a class named " Main " with a variable x:

```
public class Main {
  int x = 5;
}
```

Note: Remember that a class should always start with an uppercase first letter, and that the name of the java file should match the class name.

Create an Object

- In Java, an object is created from a class.
- We have already created the class named **Main**, so now we can use this to create objects.
- To create an object of Main, specify the class name, followed by the object name, and use the keyword **new**:

Create an object called " myObj " and print the value of x:

```
public class Main {
  int x = 5;

  public static void main(String[] args) {
    Main myObj = new Main();
    System.out.println(myObj.x);
  }
}
```

- **Multiple Objects**
- You can create multiple objects of one class:

Create two objects of Main :

```
public class Main {
  int x = 5;

  public static void main(String[] args) {
    Main myObj1 = new Main();  // Object 1
    Main myObj2 = new Main();  // Object 2
    System.out.println(myObj1.x);
    System.out.println(myObj2.x);
  }
}
```

- **Using Multiple Classes**
- You can also create an object of a class and access it in another class.
- This is often used for better organization of classes (one class has all the attributes and methods, while the other class holds the main() method (code to be executed)).

- Remember that the name of the java file should match the class name.

Main.java

```
public class Main {
  int x = 5;
}
```

Second.java

```
class Second {
  public static void main(String[] args) {
    Main myObj = new Main();
    System.out.println(myObj.x);
  }
}
```

When both files have been compiled:

```
C:\Users\Your Name>javac Main.java
C:\Users\Your Name>javac Second.java
```

Run the Second.java file:

```
C:\Users\Your Name>java Second
```

And the output will be:

```
5
```

DIFFERENCE BETWEEN CLASS AND OBJECT IN OOP

	Class	Object
Definition	Blueprint or template for creating objects	Instance of a class created using the blueprint
Usage	Used to define attributes and behavior	Used to represent a specific instance of a class
Creation	Classes are used to create objects	Objects are created from classes
Memory Allocation	Does not allocate memory for variables	Allocates memory for storing instance variables
Static Members	Can have static variables and methods	Cannot have static variables or methods
Multiple Instances	Multiple instances can be created from one class	Each object represents a separate instance
Example	```java	```java
	public class Car {	public class Car {
	private String brand;	private String brand;
	private String color;	private String color;
		public void start() {
	public void start() {	System.out.println("Car started.");

DATA MEMBERS

- In Java, "data members" are variables declared within a class but outside of any method.
- They represent the characteristics or **state** of an object created from that class.
- Data members are also commonly referred to as **fields** or **instance variables**.

- **<u>Types of data members in Java:</u>**

1) Instance Data Members (Non-Static Fields):

- These data members belong to specific instances (objects) of a class.
- Each object of the class will have its own independent copy of instance data members.
- Their memory is allocated when an object is created.
- They are accessed using an object reference (e.g., objectName.instanceVariable).

```
class Dog {
    String name; // Instance data member
    int age;     // Instance data member
}
```

2) Static Data Members (Class Variables):

- These data members belong to the class itself, not to any particular object.
- There is only one copy of a static data member, shared by all instances of the class.
- Their memory is allocated when the class is loaded into memory.
- They are accessed directly using the class name (e.g., ClassName.staticVariable), although they can also be accessed via an object reference.
- They are declared using the static keyword.

```
class Counter {
    static int count = 0; // Static data member
}
```

Methods/ Member functions

- In Java, the term "member function" is synonymous with "method."
- A method is a block of code that belongs to a class and defines the behavior of objects created from that class.
- Methods operate on the data members (variables) of an object and can also interact with other objects or perform various operations.

```
public class Main {
  static void myMethod() {
    System.out.println("Hello World!");
  }
}
```

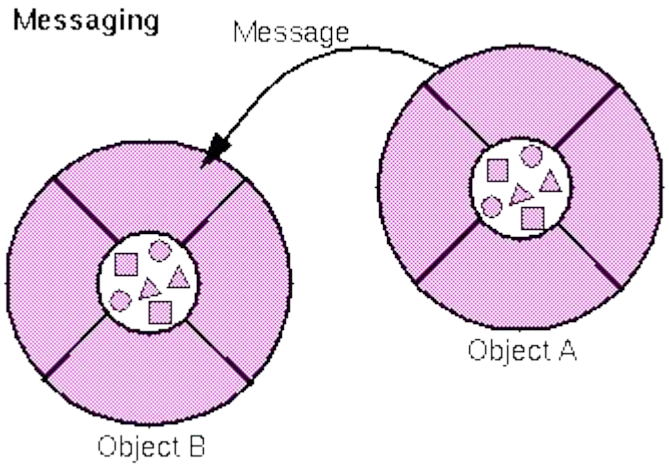
- **Declaring Data Members functions**
- Member functions, also known as methods, represent the behavior of an object.
- They perform actions and provide services related to the object's state.
- Methods can manipulate the data members of a class and perform various operations.

```
[access_modifier] [return_type] [method_name]([parameters]) {
    // Method body
}
```

```
public class Car {
  // Data members
  private String model;
  private int year;
  private double price;

  // Constructor
  public Car(String model, int year, double price) {
    this.model = model;
    this.year = year;
    this.price = price;
  }

  // Member function to display car information
  public void displayInfo() {
    System.out.println("Model: " + model);
    System.out.println("Year: " + year);
    System.out.println("Price: $" + price);
  }

  // Other methods...
}
```

Messages Passing in java

- Message Passing in terms of computers is communication between processes.
- It is a form of communication used in object-oriented programming as well as parallel programming.
- Message passing in Java is like sending an object i.e. message from one thread to another thread.
- The method call represents the message, and the arguments passed to the method are part of that message. This enables objects to communicate and interact while maintaining encapsulation.



DATA ENCAPSULATION

```
class
{

  data members
        +
  methods (behavior)

}
```

ENCAPSULATION

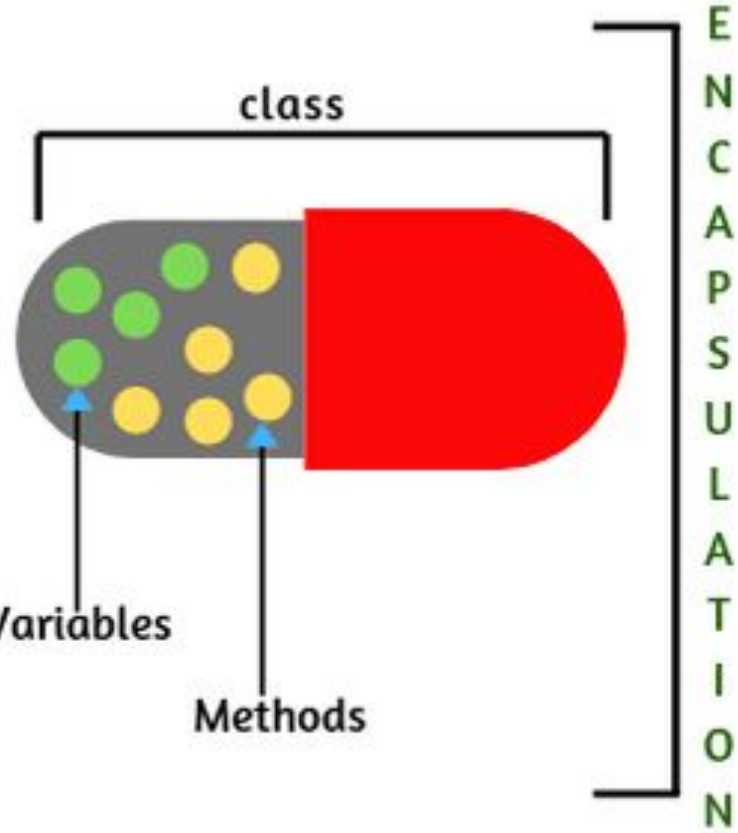


Fig: Encapsulation

- It is a mechanism that helps achieve data hiding and promotes modularity and maintainability in code.
- The meaning of **Encapsulation**, is to make sure that "sensitive" data is hidden from users.
- To achieve this, you must:
  - declare class variables/attributes as private
  - provide public **get** and **set** methods to access and update the value of a private variable.

Get and Set

- Private variables can only be accessed within the same class (an outside class has no access to it).
- However, it is possible to access them if we provide public **get** and **set** methods.
- The get method returns the variable value, and the set method sets the value.
- Syntax for both is that they start with either get or set, followed by the name of the variable, with the first letter in upper case:

Example

```java
public class Person {
  private String name; // private = restricted access

  // Getter
  public String getName() {
    return name;
  }

  // Setter
  public void setName(String newName) {
    this.name = newName;
  }
}
```

```java
public class Main {
  public static void main(String[] args) {
    Person myObj = new Person();
    myObj.setName("John"); // Set the value of the name variable to "John"
    System.out.println(myObj.getName());
  }
}

// Outputs "John"
```

Why Encapsulation?

- Better control of class attributes and methods
- Class attributes can be made **read-only** (if you only use the `get` method), or **write-only** (if you only use the `set` method)
- Flexible: the programmer can change one part of the code without affecting other parts
- Increased security of data

Data Abstraction and Information hiding:

Data Abstraction:

Data abstraction focuses on exposing only the essential functionalities and hiding the complex implementation details from the user. It allows you to design classes and interfaces that present a simplified view of an object, enabling users to interact with it without needing to understand its internal workings.

- **How it's achieved in Java:**
  - **Abstract Classes:** Classes declared with the abstract keyword can have abstract methods (methods without an implementation) and concrete methods. Subclasses must provide implementations for the abstract methods.
  - **Interfaces:** Interfaces define a contract of methods that a class must implement. They contain only abstract methods (implicitly public and abstract) and static final fields.

Information Hiding (Data Hiding):

Information hiding, often achieved through **encapsulation**, involves restricting direct access to the internal data (fields) of an object and controlling how that data is accessed and modified. This prevents unauthorized or accidental manipulation of an object's state, promoting data integrity and security.

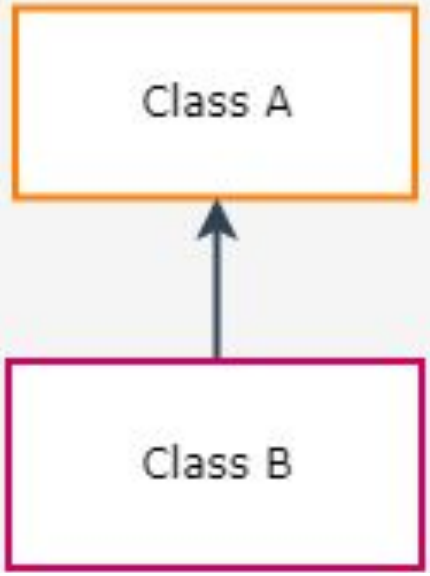
- **How it's achieved in Java:**
  - **Access Modifiers:** Java's access modifiers (private, default, protected, public) are crucial for implementing information hiding.
    - private: Members are only accessible within the same class.
    - default (package-private): Members are accessible within the same package.
    - protected: Members are accessible within the same package and by subclasses in other packages.
    - public: Members are accessible from anywhere.
  - **Getters and Setters:** Providing public "getter" methods to retrieve data and "setter" methods to modify data allows you to control the access and enforce validation rules before any changes are made.

INHERITANCE

- Inheritance in Java is a fundamental concept of Object-Oriented Programming (OOP) that allows a class to inherit properties and behaviors (fields and methods) from another class.
- This mechanism promotes code reusability, extensibility, and the creation of a hierarchical structure among classes.

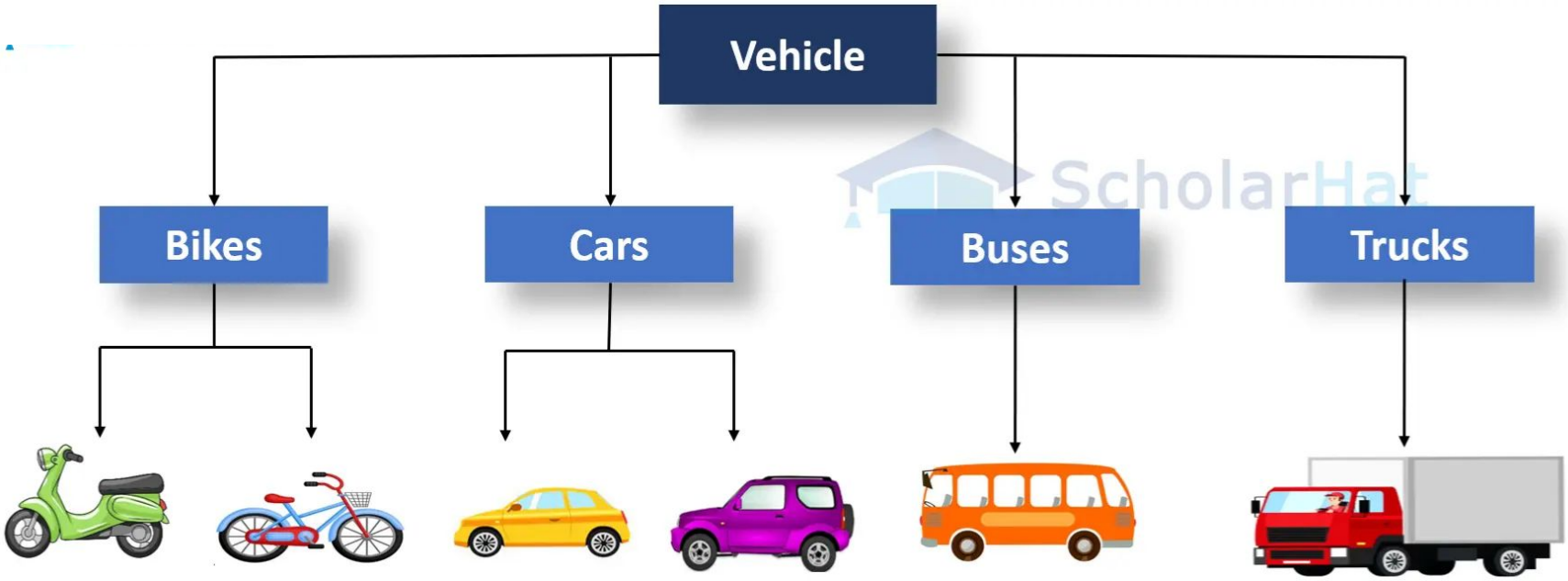
- **Key Concepts:**
- **Superclass (Parent Class / Base Class):** The class whose properties and behaviors are inherited.
- **Subclass (Child Class / Derived Class):** The class that inherits properties and behaviors from a superclass.
- **extends keyword:** Used to establish an inheritance relationship, indicating that a class is a subclass of another.

Inheritance in Java



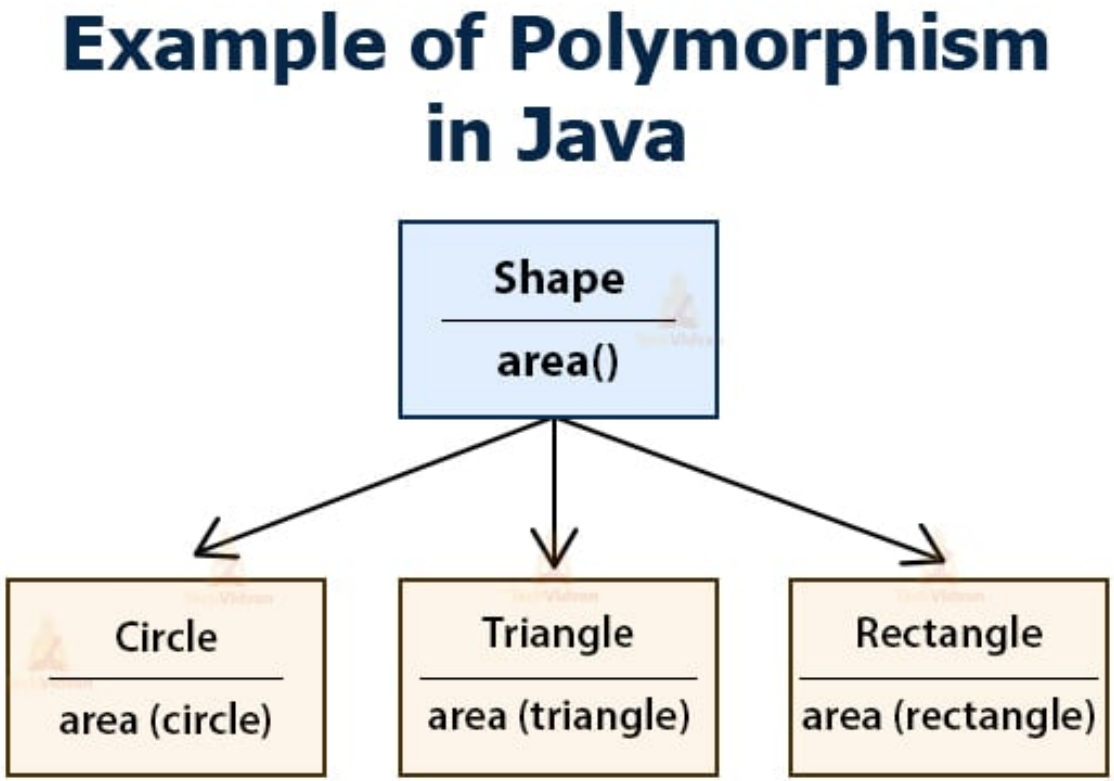

```
public class A {
    ...
}
```

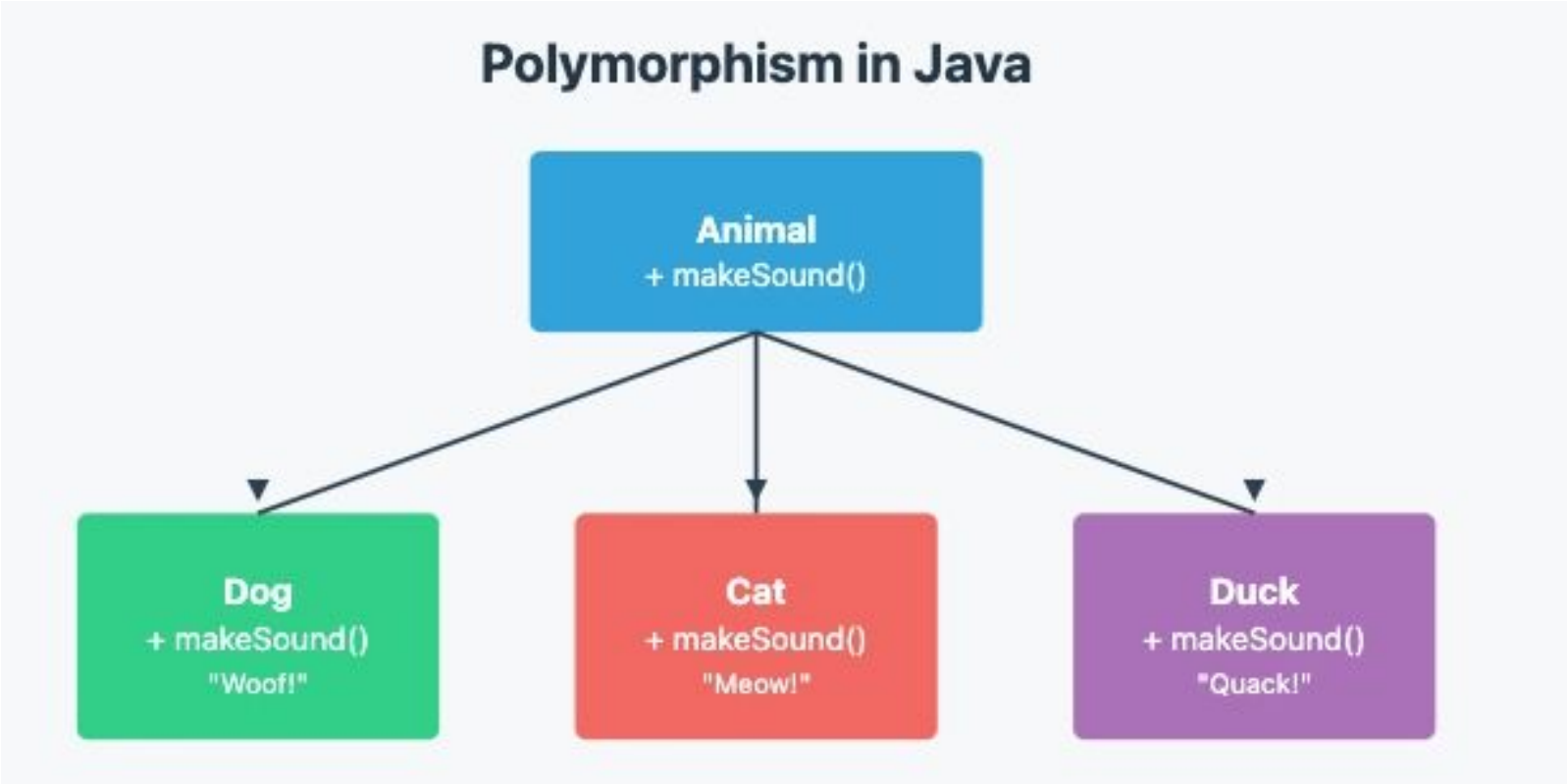
```
public class B extends A {
    ...
}
```


Vehicle
Bikes
Cars
Buses
Trucks
ScholarHat

POLYMORPHISM

- Polymorphism means "many forms", and it occurs when we have many classes that are related to each other by inheritance.




Polymorphism in Java
Animal
+ makeSound()
Dog
+ makeSound()
"Woof!"
Cat
+ makeSound()
"Meow!"
Duck
+ makeSound()
"Quack!"

Benefits of OOP

- OOP is faster and easier to execute

1. Modularity:

•OOP allows breaking down complex problems into smaller, manageable objects. This modularity makes development, testing, and maintenance easier.

2. Reusability:

- **Inheritance:** OOP enables code reuse through inheritance, allowing classes to inherit properties and methods from other classes. This reduces code duplication and speeds up development.
- **Libraries and Frameworks:** Extensive libraries and frameworks are available in Java, allowing developers to leverage existing code and functionalities, further promoting reusability.

3. Maintainability:

Well-structured OOP code is easier to understand and maintain. Changes and updates can be made more efficiently, reducing the time and effort required for maintenance.

4. Security:

- **Encapsulation:**

OOP's encapsulation feature protects data by restricting access to it, enhancing security. Only authorized methods can modify data, preventing unauthorized access and data corruption.

- **Abstraction:**

Abstraction simplifies complex systems by hiding unnecessary details, making it easier to manage and secure the system.

5. Flexibility and Scalability:

- **Polymorphism:**

Polymorphism allows objects to take on many forms, enabling flexibility in code design and making it easier to adapt to changing requirements.

- **Scalability:**

OOP principles like inheritance and modularity make it easier to scale applications as needed, handling increased data and user loads.

6. Real-World Modeling:

- OOP allows developers to model real-world entities and relationships in software, making the code more intuitive and easier to understand, especially when dealing with complex systems.

7. Enhanced Collaboration:

- OOP's clear structure and modularity facilitate collaboration among developers. Multiple developers can work on different parts of the code simultaneously, improving productivity and reducing integration issues.

Java as object oriented programming language

- Java is called object oriented because it focuses on the concept of objects:
- In Java, everything is treated as an object, allowing for better organization, reusability, and modularity of code.

Overview
of
Java Language

SIMPLE JAVA PROGRAM STRUCTURE

Structure of Java Program

Documentation Section
Package Section
Import Statements
Interface Statements
Class Definitions

main method class
{
main method definition
}

- A Java program typically consists of the following components:

1. Documentation Section
2. Package Declaration
3. Import Statements
4. Interface Section
5. Class Definition
6. Class Variables and Variables
7. Main Method Class
8. Methods and Behaviors

1) Documentation Section

- The documentation section in the structure of a Java program serves as a vital but optional part of a Java program, providing essential details about the program.
- This includes the author's name, creation date, version, program name, company name, and a brief description.
- While these details enhance the program's readability, the Java compiler ignores them during program execution.
- To include these details, programmers typically use comments.
- Comments are non-executable parts of a program. A compiler does not execute a comment.
- **Single line comment-** It is a comment that starts with // and is only used for a single line.

```
//Single-line comment
```

- **Multi line comment-** It is a comment that starts with /* and ends with */ and is used when more than one line has to be enclosed as comments.

```
/* Multiline comment
in Java */
```

- **Documentation comment-** It is a comment that starts with /** and ends with */

```
/** Documentation comment */
```

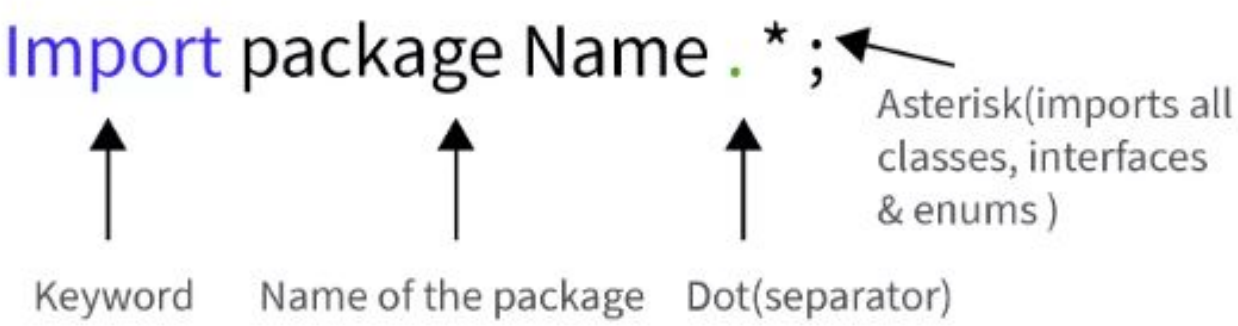
2) Package Declaration

- Declaring the package in the structure of Java is optional.
- It comes right after the documentation section.
- You mention the package name where the class belongs.
- Only one package statement is allowed in a Java program and must come before any class or interface declaration.

```
package scaler; //scaler is the package name
package com.scaler; //com is the root directory, and scaler is the sub
```

3) Import Statements

- Import statements are used to import classes, interfaces, or enums that are stored in packages or the entire package.
- A package contains many predefined classes and interfaces.
- We need to mention which package we are using at the beginning of the program.
- We do it by using the import keyword.
- We either import the entire package or a specific class from that package.



```
import java.util.*; //imports all the classes in util package
import java.util.StringTokenizer; // imports only the StringTokenizer
```

4) Interface Section

This is an optional section. The keyword interface is used to create an interface.

```
interface Code {
    void write();
    void debug();
}
```

5) Class Definition

- This is a mandatory section in the structure of Java program.
- Each Java program has to be written inside a class as it is one of the main principles of Object-oriented programming that Java strictly follows, i.e., its Encapsulation for data security.
- There can be multiple classes in a program.
- Some conventions need to be followed to name a class. They should begin with an uppercase letter.

```
class Program{
    // class definition
}
```

6) Class Variables and Variables

- Identifiers are used to name classes, methods, and variables.
- Variables are also known as identifiers in Java.
- It is a named memory location which contains a value.
- In a single statement, we're able to declare multiple variables of the same type.

```
int var=100;
int g;
char c,d; // declaring more than one variable in a statement
```

- **Rules for Naming a Variable-**
- A variable may contain characters, digits and underscores.
- The underscore can be used in between the characters.
- Variable names should be meaningful and depict the program's logic.
- Variable names should not start with a digit or a special character.

7) Main Method Class

- This is a compulsory part of the structure of Java program.
- This is the **entry point** of the compiler where the execution starts.
- It is called/invoked by the Java Virtual Machine or JVM.
- The main() method should be defined inside a class.
- We can call other functions and create objects using this method.

```
public static void main(String[] args) {
    // Method logic
}
```

8) Methods and Behaviors

- A method is a collection of statements that perform a specific task.
- Method names typically begin with a lowercase letter.

```
public void add(int a, int b){
    int c = a + b;
    System.out.println(c);
}
```

```java
import java.io.*;

public class Main{

public static void main(String[] args) throws Exception {

    System.out.println("Hello, Java!");
 }

}
```

Output-

```
Hello, Java!
```

Implementing a Java program

- Implementing a Java program involves a series of steps:

1) **Write the Java Code:** Create a .java file containing your program's source code. This includes defining classes, methods, and program logic. For example, a simple "Hello World" program would be:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

2) Compile the Java Code: Use the Java compiler (javac) to translate your source code into bytecode, which is platform-independent. Open a terminal or command prompt, navigate to the directory where your .java file is saved, and execute:

```
javac HelloWorld.java
```

This will generate a HelloWorld.class file.

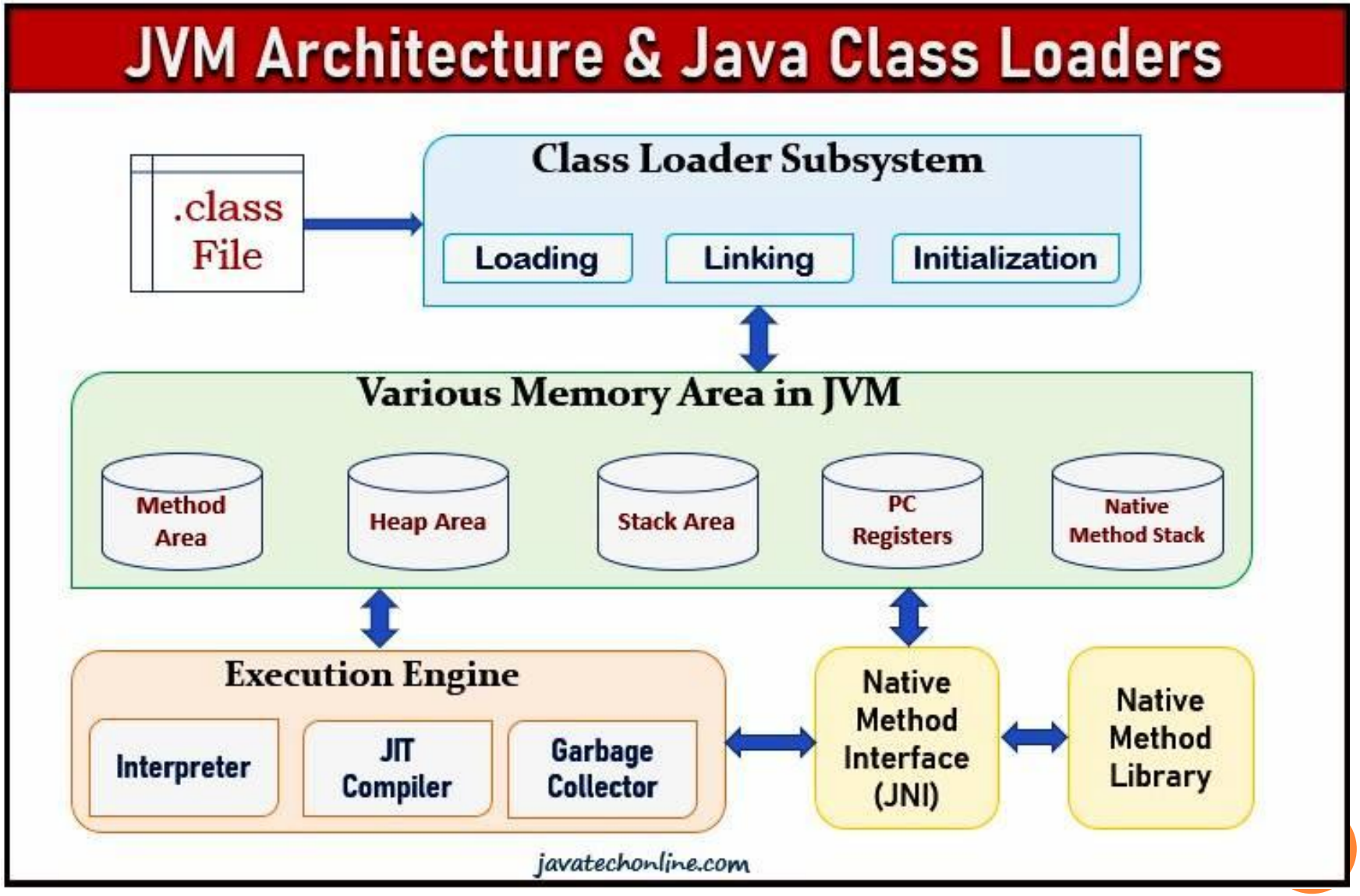
3) Run the Java Program: Execute the compiled bytecode using the Java Virtual Machine (JVM). In the same terminal or command prompt, run:

```
java HelloWorld
```

The JVM will interpret the bytecode and execute your program, displaying the output (e.g., "Hello, World!") in the console.

JVM

- The Java Virtual Machine (JVM) is an abstract machine that provides a runtime environment for executing Java bytecode.
- JVM has its own ISA, memory, stack, heap, etc. It runs on the host OS and places its demands for resources on it
- It is a core component of the Java Runtime Environment (JRE) and enables Java's "Write Once, Run Anywhere" (WORA) capability.


JVM Architecture & Java Class Loaders
.class File
Class Loader Subsystem
Loading
Linking
Initialization
Various Memory Area in JVM
Method Area
Heap Area
Stack Area
PC Registers
Native Method Stack
Execution Engine
Interpreter
JIT Compiler
Garbage Collector
Native Method Interface (JNI)
Native Method Library
javatechonline.com

Components of JVM (Java Virtual Machine) Architecture

- 1. Class Loader
- 2. Linking and Initialization
- 3. Runtime Data Areas
- 4. Execution Engine
- 5. Java Native Interface (JNI)
- 6. Native Method Libraries

1. Class Loader

- When we compile a source **(.java)** file, it gets converted into byte code as a **.class** file.
- Furthermore, when we try to use this class in our program, the class loader loads it into the main memory.
- Normally the class that contains the main() method is the first class to be loaded into the memory.
- The class loading process happens in three phases.

- **3 Phases Of Class Loading**

1) Loading

- Loading phase involves accepting the binary representation (bytecode) of a class or interface with a specific name, and generating the original class or interface from that.
- The JVM uses the ClassLoader.loadClass() method for loading the class into memory.

2) Linking

- Once a class is loaded into the memory, it undergoes the linking process.
- Linking a class or interface involves combining the different elements and dependencies of the program together.
- As the name suggests making required links with each other.

3) Initialization

- This is the final stage of class loading.
- Initialization involves executing the initialization code of the class or interface.
- This can include calling the class's constructor, executing the static block, and assigning values to all the static variables.

- **3 Class Loaders in Java**
- The class loaders load the compiled class file. Class loader system contains 3 types of class loaders.

1. Bootstrap class Loader (BCL)

- Bootstrap class Loader is responsible for loading all core java API classes. These are all classes that exist inside rt.jar which are available in all JVMs by default.
- However it's implementation is in native languages(C, C++) but not in java.
- Bootstrap Class Path is JDK/JRE/lib

2. Extension class Loader (ECL)

- Extension Class Loader is a child class of Bootstrap class loader which is responsible for loading all classes from the extension class path in java.
- The Extension Class Path is : JDK/JRE/lib/ext

3. Application class Loader (ACL)

- Application Class Loader is a child class of Extension class loader which is responsible for loading classes from application class-path.
- It's implementation is also in java.
- The Application class path is our environment class path.

How does a class loader work?

- Class loaders follow Delegation Hierarchy Principle.
- When JVM comes across a particular class, first of all it checks whether the corresponding .class file is already loaded or not.
- If it is already loaded in Method area, then the JVM considers that class is loaded without fail.
- But If it is not loaded, JVM requests class loader sub-system to load that particular class accordingly.

2. Linking and Initialization

- The linking process consists of the following three steps −

1) **Verification** − This is done by the Bytecode verifier to ensure that the generated .class files (the Bytecode) are valid. If not, an error is thrown and the linking process comes to a halt.
2) **Preparation** − Memory is allocated to all static variables of a class and they are initialized with the default values.
3) **Resolution** − All symbolic memory references are replaced with the original references. To accomplish this, the symbol table in the run-time constant memory of the method area of the class is used.

- **Initialization** is the final phase of the class-loading process. Static variables are assigned original values and static blocks are executed.

3. Runtime Data Areas

- The JVM spec defines certain run-time data areas that are needed during the execution of the program.

1) PC (Program Counter) Register

- It is local to each thread and contains the address of the JVM instruction that the thread is currently executing.

2) Stack

- It is local to each thread and stores parameters, local variables and return addresses during method calls.
- A StackOverflow error can occur if a thread demands more stack space than is permitted.
- If the stack is dynamically expandable, it can still throw OutOfMemoryError.

3) Heap

- It is shared among all the threads and contains objects, classes' metadata, arrays, etc., that are created during run-time.
- It is created when the JVM starts and is destroyed when the JVM shuts down.

4) Method Area

- This run-time area is common to all threads and is created when the JVM starts up.
- It stores per-class structures such as the constant pool (more on this later), the code for constructors and methods, method data, etc.

5) Run-Time Constant Pool

- The JVM maintains a per-class/per-type data structure that acts as the symbol table (one of its many roles) while linking the loaded classes.

6) Native Method Stacks

- When a thread invokes a native method, it enters a new world in which the structures and security restrictions of the Java virtual machine no longer hamper its freedom.

4. Execution Engine

- The execution engine is responsible for executing the bytecode, it has three different components:

1) Garbage Collection

- The JVM manages the entire lifecycle of objects in Java.
- Once an object is created, the developer need not worry about it anymore.
- In case the object becomes dead (that is, there is no reference to it anymore), it is ejected from the heap by the GC using one of the many algorithms serial GC, CMS, G1, etc.

2) Interpreter

- The interpreter Interprets the bytecode. It interprets the code fast but it's slow in execution.

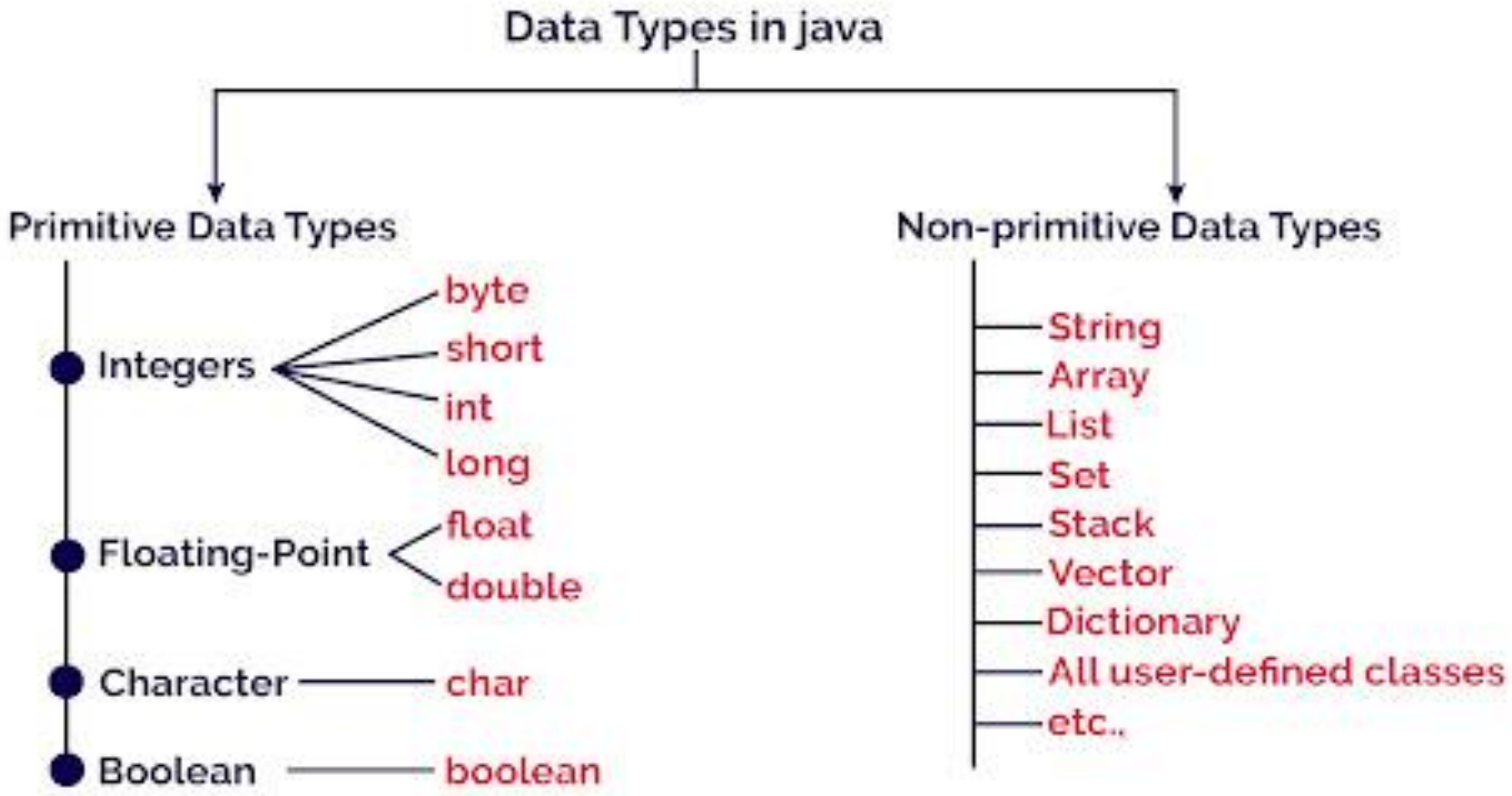
3) JIT Complier

- The JIT stands for Just-In-Time. The JIT compiler is a main part of the Java runtime environment and it compiles bytecodes to machine code at runtime.

- **5. Java Native Interface (JNI)**
- Java Native Interface (JNI) interacts with the native method libraries which are essential for the execution.

- **6. Native Method Libraries**
- Native method libraries are the collection of C and C++ libraries (native libraries) which are essential for the execution.

Data Types


Data Types in java
Primitive Data Types
Integers
byte
short
int
long
Floating-Point
float
double
Character
char
Boolean
boolean
Non-primitive Data Types
String
Array
List
Set
Stack
Vector
Dictionary
All user-defined classes
etc..

- Java categorizes data types into two main groups:
  1) Primitive data types
  2) Reference data types or Non-primitive data types

1. Primitive Data Types:

These are the most basic data types and are predefined in Java. They directly hold values and are not objects. There are eight primitive data types:

- **Integer Types:**
  - `byte`: 8-bit signed integer (range: -128 to 127)
  - `short`: 16-bit signed integer (range: -32,768 to 32,767)
  - `int`: 32-bit signed integer (most commonly used for integers)
  - `long`: 64-bit signed integer (for very large integer values)

- **Floating-Point Types:**
  - `float`: 32-bit single-precision floating-point number
  - `double`: 64-bit double-precision floating-point number (most commonly used for decimals)
- **Character Type:**
  - `char`: 16-bit Unicode character (stores a single character, enclosed in single quotes)
- **Boolean Type:**
  - `boolean`: Represents a logical value (can be either `true` or `false`)

- **2. Reference Data Types (Non-Primitive Data Types):**
- These data types do not directly store values but rather store references (memory addresses) to objects. They are created by the programmer or are built-in classes provided by Java.
- **Classes:**

  User-defined blueprints for creating objects. Examples include `String`, `Scanner`, and any custom class you define.
- **Interfaces:**

  Blueprints of a class that define a set of methods that a class must implement.
- **Arrays:**

  Collections of elements of the same data type, stored in contiguous memory locations.
- **Enums (Enumerations):**

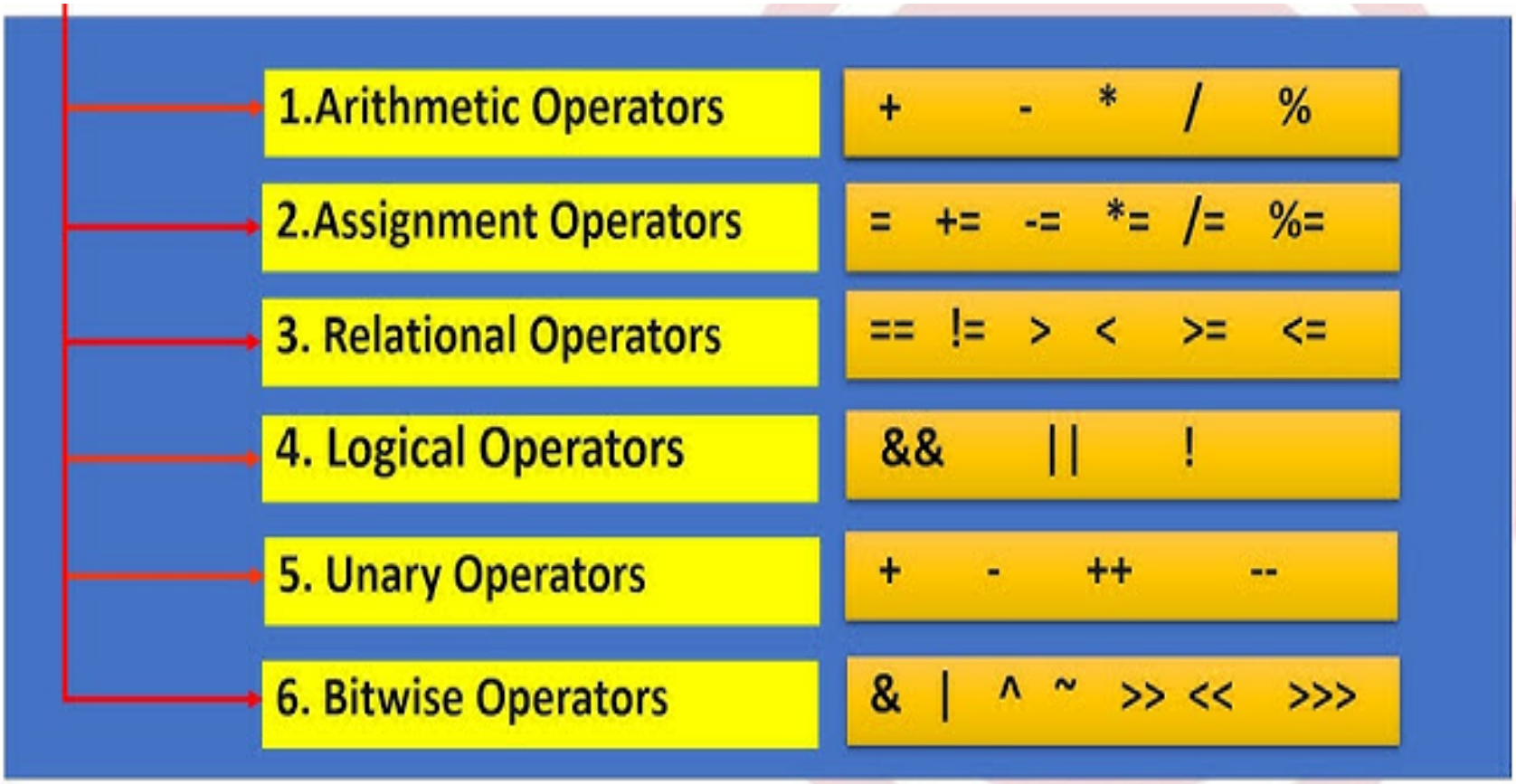
  Special types that represent a fixed set of constants.

Primitive Types vs. Reference type

Feature	Primitive Data Type	Reference Data Type
Storage	Stores actual value	Stores memory address (reference)
Memory Location	Stack	Heap (object), Stack (reference)
Size	Fixed	Variable
Methods	No	Yes
Null Assignment	No	Yes
Passing to Methods	Pass-by-value (value copied)	Pass-by-value (address copied)

Java Operators

Operators are used to perform operations on variables and values.



Expressions in Java:

- An expression is a combination of operators and operands that evaluates to a single value. Examples include:

- **Arithmetic Expressions:** Involve arithmetic operators to perform calculations (e.g., `x + y * 2`).

- **Relational Expressions:** Use relational operators to compare values, resulting in a `boolean` value (e.g., `age > 18`).

- **Logical Expressions:** Combine `boolean` values using logical operators (e.g., `(isAdult && hasLicense)`).

- **Assignment Expressions:** Assign a value to a variable using the assignment operator (`=`) (e.g., `score = 90`).

- **Conditional Expressions (Ternary Operator):** Provide a concise way to assign a value based on a condition (e.g., `String status = (isLoggedIn ? "Online" : "Offline");`).

- **Method Invocation Expressions:** Call a method and potentially use its return value (e.g., `System.out.println("Hello");`).

Operator Precedence and Associativity:

- **Precedence:**
- Determines the order in which operators are evaluated in an expression (e.g., multiplication and division have higher precedence than addition and subtraction).
- For instance, in the expression `1 + 5 * 3`, multiplication (`*`) has higher precedence than addition (`+`), so `5 * 3` is evaluated first, resulting in `15`, and then `1 + 15` is calculated, yielding `16`.

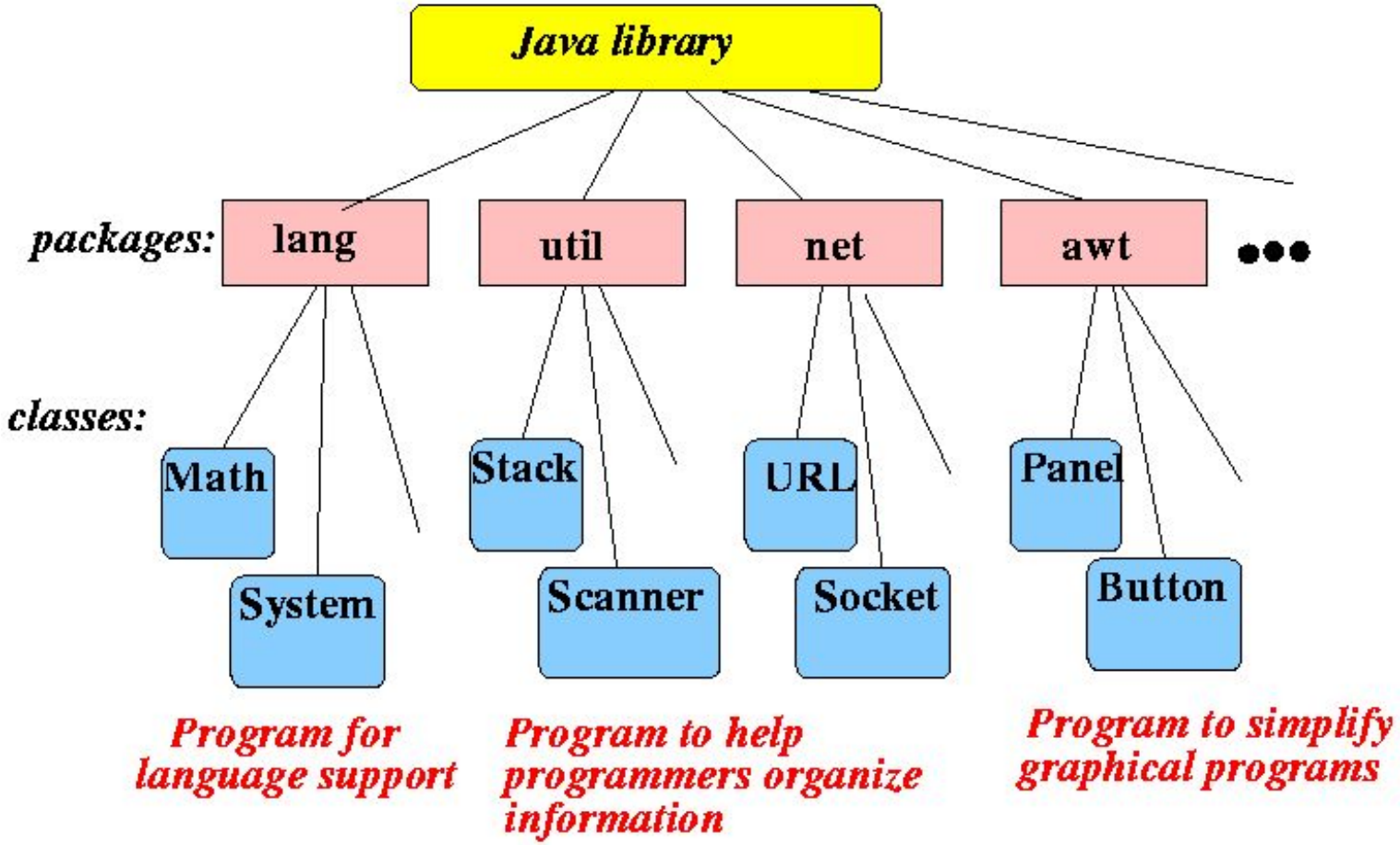
Operator's Precedence in Java

Operators	Precedence
!, +, - (unary Operators)	First (Highest)
* , / , %	Second
+ , -	Third
< , <= , >=, >	Fourth
== , !=	Fifth
&&	Sixth
||	Seventh
= (assignment Operator)	Lowest

- **Associativity:**
- Determines the order of evaluation when multiple operators of the same precedence are present (e.g., most arithmetic operators are left associative).
- Parentheses `()` can be used to explicitly control the order of evaluation, overriding default precedence and associativity.
- **Left-to-Right Associativity:** Most operators in Java, such as arithmetic operators (`+`, `-`, `*`, `/`, `%`) and relational operators (`<`, `>`, `<=`, `>=`), are left-to-right associative.
- This means they are evaluated from left to right. For example, in `a - b - c`, `(a - b)` is evaluated first, and then the result is subtracted by `c`.
- **Right-to-Left Associativity:** A few operators, notably the assignment operators (`=`, `+=`, `-=`, etc.), and unary operators (like `++`, `--`, `!`), have right-to-left associativity. For instance, in `a = b = c`, the assignment `b = c` occurs first, and then the value of `b` is assigned to `a`.

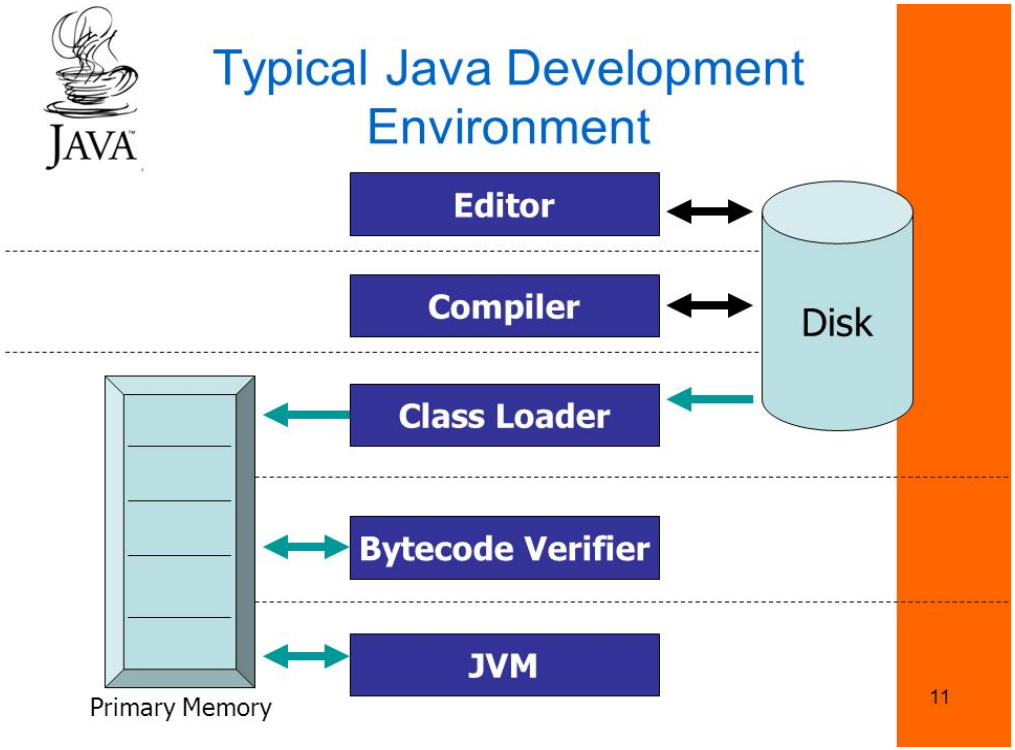
Java Class Libraries

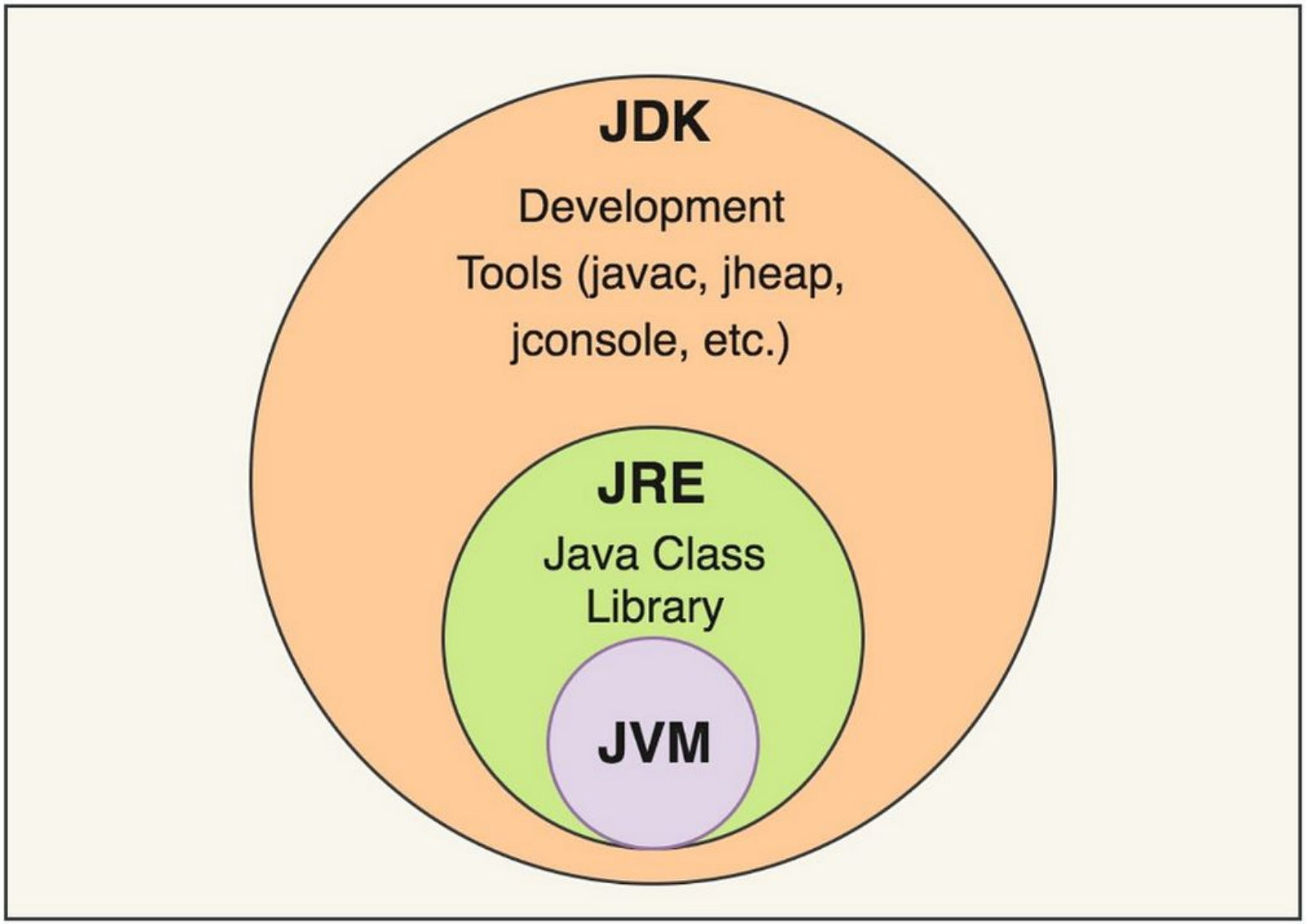
- JCL a comprehensive set of pre-built classes and interfaces that provide a wide range of functionalities for Java developers.
- These libraries are an integral part of the Java Development Kit (JDK) and are essential for building Java applications.



Typical Java Development Environment

- A typical Java development environment includes tools for writing, compiling, debugging, and running Java code, often integrated within an Integrated Development Environment (IDE).
- Key components include the Java Development Kit (JDK), an IDE, and potentially build automation tools.




JDK
Development
Tools (javac, jheap,
jconsole, etc.)
JRE
Java Class
Library
JVM

What is JDK?

- **JDK** is an abbreviation for **Java Development Kit** which includes all the tools, executables, and binaries required to compile, debug, and execute a Java Program.
- JDK is platform dependent i.e. there are separate installers for Windows, Mac, and Unix systems.
- JDK includes both JVM and JRE and is entirely responsible for code execution.

What is JRE?

- **JRE** is a **Java Runtime Environment** which is the implementation of JVM i.e. the specifications that are defined in JVM are implemented and create a corresponding environment for the execution of code.

What is JVM?

- **JVM** is the abbreviation for **Java Virtual Machine** which is a specification that provides a runtime environment in which Java byte code can be executed i.e. it is something that is abstract
- It is JVM which is responsible for converting Byte code to machine-specific code.

Memory Concepts

- The memory concept in Java revolves around how the Java Virtual Machine (JVM) manages and allocates memory for running Java applications.
- Unlike languages like C++, Java provides automatic memory management through the JVM, abstracting away the explicit allocation and deallocation of memory.
- **JVM Memory Structure**
- JVM defines various runtime data areas used during the execution of a program.
- Some of the areas are created by the JVM, whereas some are created by the threads that are used in a program.
- However, the memory area created by the JVM is destroyed only when the JVM exits.
- The data areas of a thread are created during instantiation and destroyed when the thread exits. These areas include:
  - **Heap Area**
  - **Method Area**
  - **JVM Stacks**
  - **Native Method Stacks**
  - **Program Counter (PC) Registers**

Heap Memory:

- This is the runtime data area where all objects and arrays are allocated using the `new` keyword.
- It is shared among all threads in a Java application.
- The Heap is further divided into generations (Young Generation and Old Generation) for efficient garbage collection.
- Garbage collection automatically reclaims memory from objects that are no longer referenced, preventing memory leaks.

Control Statements:

1) Selection Statements:

IF, IF-ELSE, NESTED IF-ELSE, SWITCH-CASE

2) Iteration Statements:

DO, WHILE, FOR, FOR-EACH STATEMENT,

3) break, and continue statements

Control Statements:

- Control statements in Java are used to dictate the flow of execution in a program.
- They allow developers to make decisions, repeat actions, or branch the flow of execution based on conditions.
- Here's an overview of the main types of control statements in Java:

1) Selection Statements/ Decision-Making Statements

IF, IF-ELSE, NESTED IF-ELSE, SWITCH-CASE

- **`if` Statement**: Executes a block of code if the condition is true.
- Syntax:

```
if (condition) {
    // code to execute if condition is true
}
```

- Example:

```
if (20 > 18) {
System.out.println("20 is greater than 18");
}
```

The else Statement:

- **Syntax**

```
if (condition) {
    // code to execute if condition is true
} else {
    // code to execute if condition is false
}
```

- Example

```
int time = 20;
if (time < 18) {
        System.out.println("Good day.");
 } else {
        System.out.println("Good evening.");
 }

// Outputs "Good evening."
```

The else if Statement:

- **`else if` Statement**: Executes a block of code if the condition is true.
- Syntax:

```
if (condition1) {
    // code for condition1
} else if (condition2) {
    // code for condition2
} else {
    // code if no conditions are met
}
```

- Example:

```
int time = 22;
if (time < 10) {
  System.out.println("Good morning.");
} else if (time < 18) {
  System.out.println("Good day.");
} else {
  System.out.println("Good evening.");
}
// Outputs "Good evening."
```

Java Short Hand If…Else (Ternary Operator)

- **Syntax**

```
variable = (condition) ? expressionTrue : expressionFalse;
```

Instead of writing:

```
int time = 20;
if (time < 18) {
  System.out.println("Good day.");
} else {
  System.out.println("Good evening.");
}
```

You can simply write:

```
int time = 20;
  String result = (time < 18) ? "Good day." : "Good evening.";
  System.out.println(result);
```

- `switch` statement: Provides a way to execute different blocks of code based on the value of an expression.
- <u>Syntax:</u>

```
switch (expression) {
    case value1:// code for value1
                break;
    case value2:// code for value2
                break;
    default:// code if no match
}
```

```java
int day = 4;
switch (day) {
  case 1:
    System.out.println("Monday");
    break;
  case 2:
    System.out.println("Tuesday");
    break;
  case 3:
    System.out.println("Wednesday");
    break;
  case 4:
    System.out.println("Thursday");
    break;
  case 5:
    System.out.println("Friday");
    break;
  case 6:
    System.out.println("Saturday");
    break;
  case 7:
    System.out.println("Sunday");
    break;
}
// Outputs "Thursday" (day 4)
```

2) Looping (Iteration) Statements:

- **for** loop: Executes a block of code a specific number of times.
- Syntax:
- for (initialization; condition; increment/decrement) {
  // code to repeat
  }

- Example:
- ```
  for (int i = 0; i < 5; i++) {
    System.out.println(i);
  }
  ```
- Output: 0,1,2,3,4
  ```

- `for-each` loop (Enhanced for loop): Iterates over elements of arrays or collections.

- **Syntax:**

```
for (DataType element : collection) {
    // code for each element
}
```

- **Example:**

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};

for (String i : cars) {
  System.out.println(i);
}
```

- `while` loop: Repeats a block of code as long as a condition remains true.
- **Syntax:**

```
while (condition) {
    // code to repeat
}
```

- **Example:**

```
int i = 0;
while (i < 5) {
  System.out.println(i);
  i++;
}
Output: 0,1,2,3,4
```

- `do-while` loop: Executes a block of code at least once, then repeats as long as a condition remains true.
- **Syntax:**
-
```
do {
    // code to repeat
} while (condition);
```

- **Example:**
-
```
int i = 0;
do {
  System.out.println(i);
  i++;
}
while (i < 5);
```

- **Jump (Branching) Statements:** These statements alter the normal sequential flow of execution.
  - `break` statement: Exits a loop or `switch` statement immediately.
  - ```
    for (int i = 0; i < 10; i++) {
      if (i == 4) {
        break;
      }
      System.out.println(i);
    }
    ```
  - Output: 0,1,2,3
    ```

- `continue` statement: Skips the current iteration of a loop and proceeds to the next.
- ```
  for (int i = 0; i < 10; i++) {
   if (i == 4) {
   continue;
   }
   System.out.println(i);
  }
  ```
- `Output:0,1,2,3,5,6,7,8,9`

- `return` statement: Exits the current method and optionally returns a value.
  ```

Case Study:

Analyze the object-oriented features in Java with other object oriented programming languages.

Thank You!!!